Agentic AI notes :

1. Agentic ai can take a task and complete it on its own w/o human guidance.
2. It can plan, adapts to changes and seeks help only when necessary.
3. Here chatbot is not just 'reactive'; Agentic AI is completely autonomous, it can do thing proactively.

How an AI Agent Works (Example: HR Recruitment Assistant)

1. **User Intent / Request**
   - The interaction begins with a user request.
   - Example: *"I want to hire a backend engineer."*
   - The AI agent first **understands the intent** behind the request (hire a backend engineer).

2. **Goal Formation**
   - The agent converts the request into a **clear objective**.
   - Example Goal: *Hire a remote backend engineer with 2–4 years of experience*.

3. **Planning (Task Decomposition)**
   - The agent breaks the goal into **smaller actionable steps**.
   - Example plan:
     1. Draft the Job Description (JD).
     2. Post the job on relevant hiring platforms.
     3. Monitor the candidate pipeline and adjust strategy if needed.
     4. Screen candidates and schedule interviews.
     5. Send an offer letter and begin onboarding.

4. **Execution of Actions**
   - The agent performs or triggers the required tasks.
   - This may involve using **tools, APIs, or external systems** (job boards, calendars, ATS systems).

5. **Monitoring & Feedback Loop**
   - The agent continuously **checks progress and results**.
   - If something is not working (e.g., not enough applicants), it **updates the plan**.

6. **Outcome Completion**
   - Once all steps are completed successfully, the **goal is achieved** (candidate hired and onboarded).

✅ **In simple terms:** An AI agent works by **understanding the user's goal → planning steps → executing actions → monitoring results → adjusting until the goal is achieved.**

Key Characteristics of Agentic AI

1. **Autonomous**
   - Can perform tasks **independently without constant human control**.

     Controlled Autonomy

     - Even though the agent is autonomous, humans can still control it through:
     - **Permission Scope** – defines what actions the agent can perform independently.
     - **Human-in-the-Loop (HITL)** – human approval required for certain steps.
     - **Override Controls** – humans can stop, pause, or change the agent's behavior.
     - **Guardrails / Policies** – predefined ethical or operational rules the agent must follow.

2. **Goal-Oriented**
   - Works toward achieving a **specific objective defined by the user**.

     Goals with Constraints

     - Goals may include **conditions or requirements**.
     - Example: Hire a backend engineer with **2–4 years experience**, **remote**, skills in **Python, Django, Cloud**.

     Goals Stored in Memory: The agent keeps the **goal and its progress in memory** so it can track what has been completed (e.g., JD created, job posted, interviews scheduled).

     Goals Can Change : The goal or constraints can be **updated or modified** if requirements change.

3. **Planning (most imp)**
   - Breaks a large task into **smaller steps and creates an execution plan**. Planning is basically a search problem in which we make multiple plans and search the best one....

     1. **Step 1: Generate Multiple Plans**
        - The agent creates **different possible strategies** to achieve the goal.
        - Example:
          - **Plan A:** Post job on LinkedIn, GitHub Jobs, AngelList.
          - **Plan B:** Use internal referrals or hiring agencies.
     2. **Step 2: Evaluate Each Plan**
        - The agent compares plans using criteria such as:
        - **Efficiency** – which option is faster.
        - **Tool Availability** – which tools or platforms are available.
        - **Cost** – whether the option requires paid tools.
        - **Risk** – possibility of failure or low results.
        - **Alignment with Constraints** – e.g., remote hiring or limited budget.
     3. **Step 3: Select the Best Plan**
        - The agent chooses the best strategy using:
        - **Human-in-the-loop input** (human chooses preferred option), or
        - **Predefined policies** (e.g., prioritize low-cost options first).

4. **Reasoning**
   - Uses logical thinking to **decide what action to take next**.

1. Reasoning During Planning

- **Goal Decomposition** – Break a large goal into smaller actionable steps.
- **Tool Selection** – Decide which tools or APIs are needed for each step.
- **Resource Estimation** – Estimate time, dependencies, and possible risks.

2. Reasoning During Execution

- **Decision Making** – Choose the best option during task execution (e.g., select best candidates).
- **Human-in-the-Loop Handling (HITL)** – Ask for human help when uncertain.
- **Error Handling** – Detect failures (e.g., API errors) and recover or retry.

5. **Adaptability**
   - Can **adjust its strategy** if conditions change or if the initial plan fails.

6. **Context Awareness**
   - Understands the **current situation, past interactions, and environment** to make better decisions.

Types of Context

1. **Original Goal** – The main objective the agent is trying to achieve.
2. **Progress / Interaction History** – Tracks what has already been done (e.g., JD created and posted).
3. **Environment State** – External conditions affecting the task (e.g., number of applicants).
4. **Tool Responses** – Results from tools/APIs (e.g., resume parser output).
5. **User Preferences** – Specific user choices (e.g., prefers remote candidates).
6. **Policies / Guardrails** – Rules that restrict agent actions.

How Context Awareness is Implemented

- Memory Systems

1. Short-Term Memory
   - Stores **current task information and recent interactions.**
2. Long-Term Memory
   - Stores **past knowledge, preferences, and historical data** for future use.

<u>Components of an Agentic AI System</u>

1. **Brain:** The core **LLM or reasoning engine** that understands tasks, plans actions, and makes decisions.
2. **Orchestrator:** Manages the **workflow of the agent**, deciding which step to run next and coordinating different components. LangGraph is the orchestrator here.
3. **Tools:** External **APIs, databases, or services** the agent uses to perform tasks (e.g., job boards, resume parsers).
4. **Memory:** Stores **task progress, past interactions, and user preferences** for better decision-making.
5. **Supervisor:** Provides **oversight and control**, ensuring the agent follows rules, policies, or human approvals when needed. Basically Guardrails enforcement.

Why Does LangGraph Exist?

LangChain Components : https://aws.amazon.com/what-is/langchain/

1. Model Component
    - Provides a unified interface to interact with different LLM providers (e.g., OpenAI, Anthropic, etc.).
2. Prompts Component
    - Helps design and manage prompts used to guide the LLM's responses.
3. Retrievers Component
    - Retrieves relevant documents or information from a vector database.
4. **Chains (Key Feature)**
    - Connects multiple components together to create a sequence of steps for completing tasks.

Workflows vs Agents : https://www.anthropic.com/engineering/building-effective-agents

1. Basically workflows are less autonomous because their flow/design is static like the agent has to follow the rules/paths/tools we pre-define.
2. Whereas in agents -> an agent has full autonomy, it can decide what path to take, which tool to use dynamically based on the reasoning it comes up with.

Challenges in langchain

When workflows become bigger, the agent does not just move in a straight line.

It starts needing:

1. **Conditional branches**
    - "If candidate is good, schedule interview."
    - "Else, reject."
2. **Loops**
    - "Keep searching until enough candidates are found."
    - "Retry tool call until success."
3. **Jumps / dynamic routing**
    - The next step is not fixed.
    - The system may jump to a different tool or step depending on results.

So the issue is:

> **Simple chains work well for linear tasks, but real agent workflows are not linear.**

They are more like **decision trees / graphs**, not straight pipelines.

Example

Suppose the user says:

"Hire a backend engineer."

A simple chain might be:

Draft JD → Post Job → Screen Candidates → Schedule Interviews

But in reality:

- If JD is incomplete → rewrite JD
- If not enough applicants → repost on more platforms
- If candidate quality is poor → change screening criteria
- If salary issue arises → ask human
- If API fails → retry or use another tool

Now the workflow is no longer one straight chain.

LangChain chains are good for fixed linear workflows, but agentic systems need branching, looping, and dynamic routing, which makes control flow complex. That's why we make graphs because they are good with dealing with non-linear structures.

 # challenge - 2 : State Handling

What is State in LangGraph?

State in LangGraph is a way to maintain and track information as an AI system processes data. Think of it as the system's memory, allowing it to remember and update information as it moves through different stages of a workflow, or graph.

State exists in key-value pairs :

```
state = {
  goal: "Hire a backend software engineer",

  jd: "",
  jd_approved: false,
  jd_posted: false,

  min_applicants: 5,
  num_applications: 0,

  shortlisted_candidates: {},

  interview_questions: [],

  offer_status: {
      sent: false,
      accepted: false,
      renegotiated: false
  },

  onboarding_status: {
      completed: false,
      start_date: None,
      employee_id: None
  }
}
```

Langchain mai state ko hum map nahi krskte...cuz state is in key-value pairs, langchain doesn't give any options to store things in any key-value pairs. Langchain has a memory concept - conversational memory concept in which our chat with LLM can be stored as memory. Langchian mai state manually banana [dega aur hard rules likhne pdege to update state at certain steps.

So basically langGraph is :

1. **Orchestration Framework**
   - LangGraph is a framework used to **orchestrate and manage multi-step LLM workflows** that are intelligent and stateful.
2. **Advanced Workflow Features**
   - Supports complex behaviors such as:
   - **Parallelism** – multiple tasks running at the same time
   - **Loops** – repeating steps until a condition is met
   - **Branching** – different paths depending on results
   - **Memory** – storing information across steps
   - **Resumability** – workflows can pause and continue later
3. **Graph-Based Architecture**
   - Instead of a **linear chain of steps**, LangGraph models workflows as a **graph**:
   - **Nodes** → tasks or operations
   - **Edges** → routes that decide which step runs next

Example state variables:

```
essay_text : str
topic : str
depth_score : int
language_score : int
clarity_score : int
total_score : int
feedback : list[str]
evaluation_round : int
```

Each node:

```
reads state → updates state → passes it forward
```

```
Generate Topic
        ↓
Write Essay
        ↓
 ┌──────────────┬──────────────┬──────────────┐
 ↓              ↓              ↓
Clarity       Depth          Language
Evaluation    Evaluation     Evaluation
 └──────────────┴──────────────┴──────────────┘
                ↓
         Final Evaluation
           /        \
          /          \
     Success       Give Feedback
        ↓               ↓
       END          Revise Essay
                        ↓
                    Re-evaluate
```

Workflow in LangGraph:

LangGraph Execution Model (Brief Notes)

1. Graph Definition

First, you define the **structure of the workflow**:

- **State schema** → the shared data structure used by the workflow
- **Nodes** → functions that perform tasks
- **Edges** → connections that define which node runs next

Example:

```
Node A → Node B → Node C
```

---

2. Compilation

After defining the graph, you **compile it**.

```
graph.compile()
```

Purpose:

- Checks if the graph structure is valid
- Prepares the workflow for execution

Think of this like **building the workflow engine before running it**.

3. Invocation

Next, you **run the graph** by providing an **initial state**.

```
graph.invoke(initial_state)
```

LangGraph sends the **initial state as a message** to the **starting node** of the graph.

Example initial state:

```
{
  topic: "Climate Change",
  essay_text: ""
}
```

---

4. Super-Steps Begin

Execution then proceeds in **rounds called super-steps**.

In each super-step:

- Nodes receive the **current state**
- Nodes perform their **task**
- Nodes **update the state**
- The updated state is passed to the **next nodes**

Parallel nodes can run **simultaneously** in the same super-step.

5. Message Passing & Node Activation

- Nodes communicate by **passing messages through edges**.
- When a node finishes its task, it sends the **updated state/message to the next nodes**.
- Any node that **receives a message becomes active in the next round (super-step)**.

Example flow:

```
Node A finishes
     ↓
Message sent to Node B and Node C
     ↓
Node B and Node C activate next
```

This allows **parallel execution** if multiple nodes receive messages.

---

6. Halting Condition

The workflow **stops executing** when:

1. **No nodes are active**
2. **No messages are in transit**

Meaning:

- No more tasks need to run
- The workflow has reached its **final state**